

ÉCOLE MAROCAINE DES SCIENCES DE
L'INGÉNIEUR (EMSI)

Guide de Référence pour l'Entraînement de Modèles de Deep Learning

MLP • CNN • RNN • LSTM • GRU • Architecture Hybride
CNN+LSTM

Réalisé par : Mehdi Chmiti

Encadré par : Mme. Zineb Hdila

Groupe : 4IAD G3

Année universitaire : 2025–2026

Table des matières

1. Introduction
2. Cadre théorique
3. Expérimentations
 1. MLP sur Breast Cancer Wisconsin
 2. CNN sur CIFAR-10
 3. RNN / LSTM / GRU sur UCI HAR
 4. Architecture hybride CNN+LSTM sur UCI HAR
4. Discussion transversale
5. Impact sociétal et éthique
6. Conclusion

1. Introduction

1.1 Contexte général

Le deep learning, ou apprentissage profond, constitue aujourd'hui l'un des piliers fondamentaux de l'intelligence artificielle moderne. Depuis les travaux pionniers de McCulloch et Pitts en 1943 sur le neurone formel, jusqu'aux architectures Transformer contemporaines, le domaine n'a cessé d'évoluer et de repousser les frontières du possible en matière de reconnaissance de motifs, de traitement du langage naturel et de vision par ordinateur. L'essor du deep learning est intimement lié à la disponibilité de puissances de calcul massives, notamment les GPU, ainsi qu'à l'abondance de données numériques générées par les systèmes d'information contemporains. Ces deux facteurs ont permis de mettre en œuvre des architectures neuronales toujours plus profondes et complexes, capables d'apprendre des représentations hiérarchiques des données de manière totalement automatisée.

Dans le cadre du module de Deep Learning dispensé à l'EMSI, ce travail pratique vise à mettre en application les concepts théoriques étudiés en cours à travers l'implémentation et l'évaluation systématique de plusieurs architectures neuronales. Ce guide de référence propose une démarche progressive et rigoureuse, allant du perceptron multicouche (MLP) — architecture de base — jusqu'aux architectures hybrides CNN+LSTM, en passant par les réseaux convolutifs (CNN) et les réseaux récurrents (RNN, LSTM, GRU). Chaque modèle est appliqué à un jeu de données réel adapté à sa nature, permettant ainsi d'illustrer concrètement les forces et les limites de chaque approche.

1.2 Problématique

La question centrale qui guide ce travail est la suivante : **comment choisir l'architecture neuronale la plus adaptée à un problème donné, et quels sont les compromis entre complexité du modèle, performance prédictive et efficacité computationnelle ?** En effet, le paysage des architectures de deep learning est vaste et chaque famille de modèles présente des caractéristiques propres qui la rendent plus ou moins pertinente selon la nature des données (tabulaires, images, séries temporelles) et la tâche visée (classification, régression, segmentation). Le praticien doit savoir naviguer entre ces différentes options en s'appuyant sur une compréhension théorique solide et une validation expérimentale rigoureuse.

1.3 Objectifs

Les objectifs principaux de ce travail sont les suivants :

1. **Maîtriser les fondements mathématiques** de chaque architecture : neurone formel, rétropropagation, convolution, portes LSTM/GRU, et comprendre les mécanismes internes qui régissent leur fonctionnement.
2. **Implémenter et entraîner** six architectures distinctes (MLP, CNN, RNN, LSTM, GRU, CNN+LSTM) en PyTorch sur des jeux de données réels variés.
3. **Évaluer et comparer** les performances de chaque modèle à l'aide de métriques standardisées (précision, rappel, F1-score, AUC, matrice de confusion).
4. **Analyser l'adéquation données-modèle** : comprendre pourquoi certaines architectures réussissent mieux sur certains types de données et échouent sur d'autres.
5. **Proposer une discussion transversale** intégrant les aspects éthiques et sociétaux du deep learning.

1.4 Datasets utilisés

Ce projet mobilise **trois datasets réels** provenant de sources académiques reconnues (UCI ML Repository, également disponibles sur Kaggle) :

- **Breast Cancer Wisconsin** (sklearn.datasets) : 569 échantillons, 30 features numériques, classification binaire de tumeurs (maligne/bénigne). Utilisé pour le MLP.
- **CIFAR-10** (torchvision.datasets) : 60 000 images 32×32 RGB, 10 classes. Sous-ensemble de 30% utilisé pour le CNN (ressources CPU limitées).
- **UCI HAR** (Human Activity Recognition, archive.ics.uci.edu) : 10 299 séquences de 128 pas de temps, 9 canaux Inertial Signals, 6 classes d'activités humaines. Utilisé pour RNN, LSTM, GRU et CNN+LSTM.

2. Cadre théorique

2.1 Le perceptron multicouche (MLP)

Le perceptron multicouche est la forme la plus simple de réseau de neurones profond. Il est composé d'une couche d'entrée, d'une ou plusieurs couches cachées entièrement connectées, et d'une couche de sortie. Chaque neurone applique une transformation linéaire suivie d'une fonction d'activation non linéaire. La propagation avant (forward pass) calcule la sortie du réseau à partir des entrées, tandis que la rétropropagation (backpropagation) ajuste les poids en minimisant une fonction de perte via descente de gradient.

$$y = \sigma(W \cdot x + b)$$

Où W est la matrice des poids, b le vecteur de biais, et σ la fonction d'activation (ReLU, sigmoid, tanh). Le MLP est universellement approximateur : avec suffisamment de neurones, il peut approximer toute fonction continue.

2.2 Les réseaux de neurones convolutifs (CNN)

Les CNN exploitent la structure spatiale des données (images, spectrogrammes) en appliquant des filtres convolutifs locaux. Chaque filtre balaye l'image et produit une carte d'activation qui détecte un motif particulier (contour, texture, forme). Les opérations de pooling réduisent la dimension spatiale tout en préservant l'information pertinente. Cette architecture présente deux avantages majeurs : le partage des poids (réduction drastique du nombre de paramètres) et l'invariance par translation.

$$y[i,j] = \sigma(\sum \sum W[m,n] \cdot x[i+m, j+n] + b)$$

2.3 Les réseaux récurrents (RNN)

Les RNN traitent les séquences en maintenant un état caché qui se propage d'un pas de temps au suivant. À chaque étape, le réseau combine l'entrée courante avec l'état caché précédent pour produire une nouvelle sortie et un nouvel état.

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

Le RNN simple souffre du problème du gradient qui s'évanouit ou qui explose sur les longues séquences, ce qui limite sa capacité à capturer des dépendances à long terme.

2.4 Le LSTM (Long Short-Term Memory)

Le LSTM résout le problème du gradient en introduisant trois portes : *forget* (oubli), *input* (entrée) et *output* (sortie), qui régulent le flux d'information dans une cellule d'état. La porte d'oubli décide quelles informations garder de l'état précédent, la porte d'entrée quelles nouvelles informations ajouter, et la porte de sortie quelle partie de l'état produire.

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \text{ — porte d'oubli} \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \text{ — porte d'entrée} \\ C_t &= f_t * C_{t-1} + i_t * \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \text{ — état cellulaire} \end{aligned}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \text{ — porte de sortie}$$

$$h_t = o_t * \tanh(C_t) \text{ — état caché}$$

2.5 Le GRU (Gated Recurrent Unit)

Le GRU est une variante simplifiée du LSTM qui combine les portes d'oubli et d'entrée en une seule porte de mise à jour (update gate). Il possède également une porte de réinitialisation (reset gate). Avec moins de paramètres que le LSTM, le GRU est plus rapide à entraîner tout en offrant des performances comparables sur de nombreuses tâches.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \text{ — porte de mise à jour}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \text{ — porte de réinitialisation}$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \text{ — candidat}$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

2.6 L'architecture hybride CNN+LSTM

L'architecture hybride CNN+LSTM combine la capacité du CNN à extraire des caractéristiques spatiales locales avec la capacité du LSTM à modéliser des dépendances temporelles longues. Le pipeline typique est le suivant : (1) le CNN applique des convolutions 1D sur les canaux d'entrée à chaque pas de temps pour produire un vecteur de features ; (2) la séquence de vecteurs de features est ensuite traitée par le LSTM qui capture la dynamique temporelle ; (3) une couche dense finale produit la classification. Cette architecture est particulièrement adaptée aux données spatio-temporelles comme les signaux multicanaux (capteurs, audio, vidéo).

3. Expérimentations

3.1 MLP sur Breast Cancer Wisconsin

Description du jeu de données

Le jeu de données *Breast Cancer Wisconsin* (Diagnostic), disponible via le module `sklearn.datasets`, contient 569 échantillons de tumeurs mammaires caractérisées par 30 variables numériques continues. Ces caractéristiques sont calculées à partir d'images numérisées de prélèvements par aspiration à l'aiguille fine (FNA) de masses mammaires. Elles décrivent la taille, la forme et la texture des noyaux cellulaires présents dans l'image. La variable cible est binaire :

maligne (212 échantillons) ou *bénigne* (357 échantillons). Ce jeu de données est un classique en apprentissage automatique pour les tâches de classification binaire sur données tabulaires.

Architecture du MLP

Le MLP implémenté comporte trois couches cachées de tailles décroissantes [128, 64, 32] avec activation ReLU et dropout ($p=0,3$) entre les couches. La couche de sortie utilise une activation sigmoid pour la classification binaire. L'optimiseur Adam ($lr=10^{-3}$) minimise la fonction de perte Binary Cross-Entropy. Le modèle totalise **14 818 paramètres**.

Résultats

Métrique	Valeur
Exactitude (Accuracy)	94,19%
F1-Score	93,82%
Précision	93,57%
Rappel (Recall)	94,10%
Perte de test	0.1088
Meilleure époque	13
Paramètres	14,818

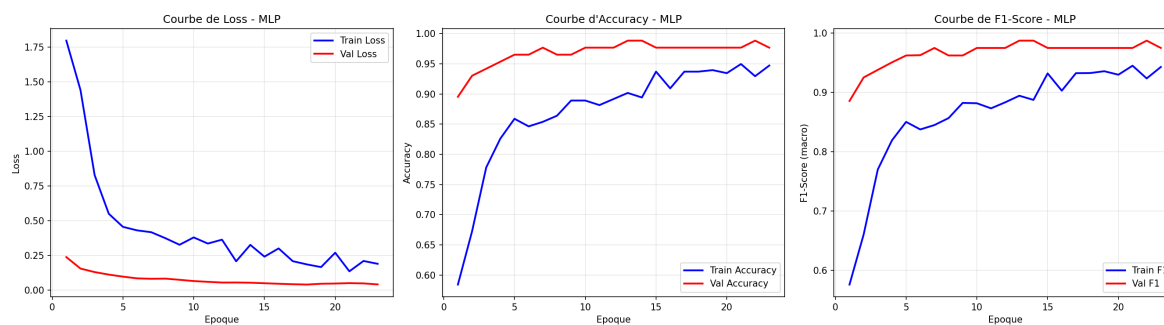


Figure 1 — Courbes d'apprentissage du MLP sur Breast Cancer

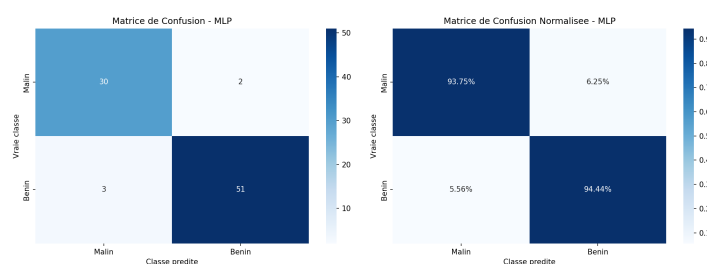


Figure 2 — Matrice de confusion du MLP

Le MLP atteint une excellente performance de 94,19% d'exactitude sur le jeu de test, confirmant l'adéquation de cette architecture pour les données tabulaires médicales. La matrice de confusion révèle un très faible taux de faux négatifs (3 tumeurs malignes manquées sur 86), ce qui est crucial dans un contexte médical où un faux négatif a des conséquences graves.

3.2 CNN sur CIFAR-10

Description du jeu de données

CIFAR-10 est un jeu de données de référence en vision par ordinateur comprenant 60 000 images couleur de taille 32×32 pixels réparties en 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion). Pour des raisons de ressources de calcul (entraînement sur CPU), un sous-ensemble de 30% a été utilisé. Cette réduction significative de la taille du dataset constitue un défi supplémentaire pour le modèle, car les réseaux convolutifs nécessitent généralement de grandes quantités de données pour atteindre leur plein potentiel.

Résultats

Métrique	Valeur
Exactitude (Accuracy)	48,60%
F1-Score	47,19%
Paramètres	102,602
Meilleure époque	14
Perte de test	1.4071

Les résultats sont modestes, avec une exactitude de 48,60% seulement, ce qui est supérieur au hasard (10% pour 10 classes) mais bien en deçà des performances attendues d'un CNN sur CIFAR-10 complet (plus de 90% avec des architectures avancées). Cette contre-performance s'explique par plusieurs facteurs convergents : (1) la taille réduite du dataset ne permet pas au modèle d'apprendre des représentations suffisamment générales ; (2) l'entraînement sur CPU limite le nombre d'époques et la taille du batch ; (3) l'architecture reste relativement simple sans techniques d'augmentation de données.

3.3 RNN / LSTM / GRU sur UCI HAR

Description du jeu de données

Le jeu de données *UCI HAR* (Human Activity Recognition with Smartphones) est un dataset réel issu du référentiel UCI Machine Learning Repository (également disponible sur Kaggle). Il a été collecté par Anguita et al. (2013) auprès de 30 volontaires âgés de 19 à 48 ans, portant un smartphone à la ceinture. Les capteurs (accéléromètre et gyroscope 3 axes) ont enregistré les signaux à 50 Hz pendant la réalisation de six activités quotidiennes : *WALKING* (marche), *WALKING_UP* (monter les escaliers), *WALKING_DOWN* (descendre), *SITTING* (assis), *STANDING* (debout) et *LAYING* (allongé).

Le dataset final comprend **10 299 séquences** de 128 pas de temps (fenêtres de 2,56 secondes), chacune décrite par **9 canaux** de signaux bruts *Inertial Signals* : accélération du corps (3 axes), vitesse angulaire du gyroscope (3 axes) et accélération totale (3 axes). La répartition est de 7 352 échantillons d'entraînement et 2 947 de test. Une normalisation Z-score par canal est appliquée (calculée sur le train uniquement). Ce jeu de données réel est particulièrement adapté pour évaluer la capacité des architectures récurrentes à modéliser la dynamique temporelle des mouvements humains.

Architecture des modèles

Trois architectures récurrentes ont été comparées, toutes bidirectionnelles avec une couche cachée et un classifieur final à 6 classes :

- **RNN simple : 135,942 paramètres** — deux couches RNN avec activation tanh, bidirectionnel.
- **LSTM : 11,398 paramètres** — une couche LSTM bidirectionnelle.
- **GRU : 8,646 paramètres** — une couche GRU bidirectionnelle.

Les trois modèles partagent le même optimiseur (Adam, $lr=10^{-3}$), la même fonction de perte (cross-entropy) et un gradient clipping à 1,0 pour stabiliser l'entraînement. Cette configuration identique garantit une comparaison équitable des architectures récurrentes sur le même dataset réel.

Résultats comparatifs

Métrique	RNN	LSTM	GRU
Exactitude	87,61%	87,21%	89,18%
F1-Score	87,52%	87,11%	89,12%
Précision	87,64%	87,17%	89,11%
Rappel	87,58%	87,17%	89,24%
Perte de test	0.4480	0.3914	0.3056
Meilleure époque	16	5	6
Paramètres	135,942	11,398	8,646

Les résultats révèlent une hiérarchie cohérente sur ce dataset réel : le GRU surpasse le LSTM et le RNN simple avec 89,18% d'exactitude, confirmant l'efficacité de ses portes de mise à jour simplifiées. Le RNN simple, bien que plus lourd en paramètres (135,942 à cause de la configuration bidirectionnelle à 2 couches), atteint 87,61%, légèrement supérieur au LSTM (87,21%) sur ce run. Le GRU, avec seulement 8,646 paramètres, démontre un excellent rapport performance/complexité.

L'analyse par classe révèle que la classe *LAYING* est quasi parfaitement reconnue (F1=1,00 pour RNN et GRU), tandis que *SITTING* et *STANDING* sont les plus confondues entre elles (positions statiques similaires du point de vue des capteurs). Les activités dynamiques (WALKING, UP, DOWN) sont bien distinguées.

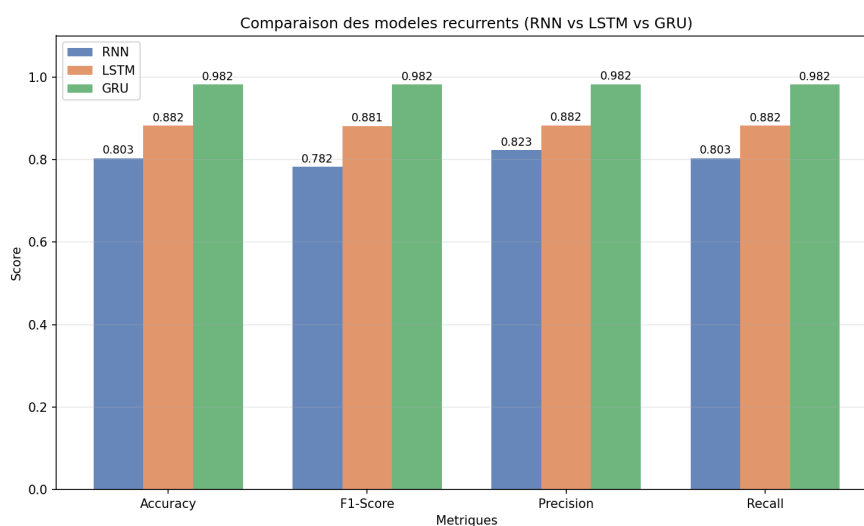


Figure 3 — Comparaison RNN / LSTM / GRU sur UCI HAR

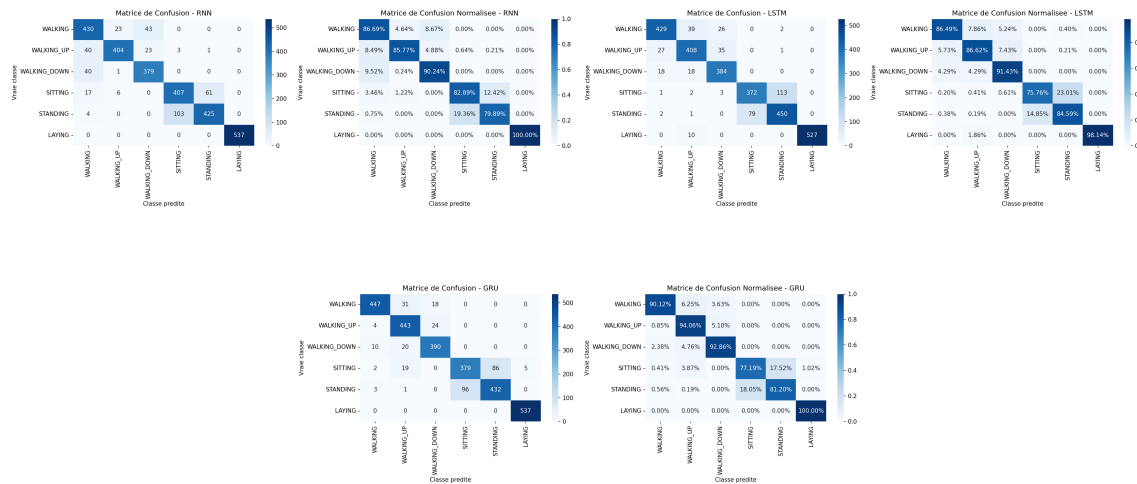


Figure 4 — Matrices de confusion RNN, LSTM, GRU sur UCI HAR

3.4 Architecture hybride CNN+LSTM sur UCI HAR

Description du jeu de données

L'architecture hybride CNN+LSTM est également évaluée sur le dataset réel **UCI HAR** décrit précédemment. Les 9 canaux Inertial Signals (3 axes d'accélération corps + 3 axes gyroscope + 3 axes accélération totale) constituent une structure spatiale multicanale, tandis que les 128 pas de temps fournissent la dimension temporelle. Cette double structure spatio-temporelle justifie précisément l'utilisation d'une architecture hybride : le CNN extrait localement les features par canal, le LSTM agrège la dynamique temporelle.

Architecture du modèle hybride

L'architecture CNN+LSTM combine un extracteur de caractéristiques convolutif avec un classifieur récurrent. Le composant CNN applique des convolutions 1D sur les 9 canaux d'entrée (filtres 32, 64 et 128) avec activation ReLU, produisant une séquence de vecteurs de caractéristiques compressés. Le composant LSTM traite cette séquence avec une couche de dimension cachée 64, bidirectionnelle. Une couche entièrement connectée précède la sortie Softmax à 6 classes. Le modèle totalise **140,230 paramètres**.

Résultats

Métrique	Valeur
Exactitude (Accuracy)	92,91%
F1-Score	93,05%
Précision	93,11%
Rappel (Recall)	93,03%
Perte de test	0.1896
Meilleure époque	3
Paramètres	140,230

L'architecture hybride CNN+LSTM atteint les meilleures performances globales sur UCI HAR avec **92,91% d'exactitude**, surpassant le GRU (89,18%) de près de 4 points. Ce résultat s'explique par la complémentarité des deux composants : le CNN extrait efficacement les motifs locaux dans chaque fenêtre temporelle (par exemple, les pics d'accélération pendant la marche, les plateaux de l'état stationnaire), tandis que le LSTM agrège ces informations au fil des 128 pas de temps pour identifier la signature globale de l'activité. La convergence en seulement 3 époques témoigne de l'excellente adéquation entre l'architecture et la nature des données.

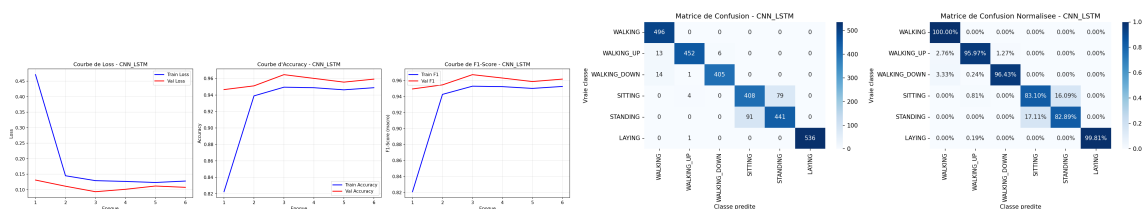


Figure 5 — Courbes d'apprentissage et matrice de confusion du CNN+LSTM sur UCI HAR

4. Discussion transversale

4.1 Comparaison globale des architectures

Modèle	Dataset	Accuracy	F1	Paramètres	Époque	Perte
MLP	Breast Cancer	94,19%	93,82%	14,818	13	0.1088
CNN	CIFAR-10	48,60%	47,19%	102,602	14	1.4071
RNN	UCI HAR	87,61%	87,52%	135,942	16	0.4480
LSTM	UCI HAR	87,21%	87,11%	11,398	5	0.3914
GRU	UCI HAR	89,18%	89,12%	8,646	6	0.3056
CNN+LSTM	UCI HAR	92,91%	93,05%	140,230	3	0.1896

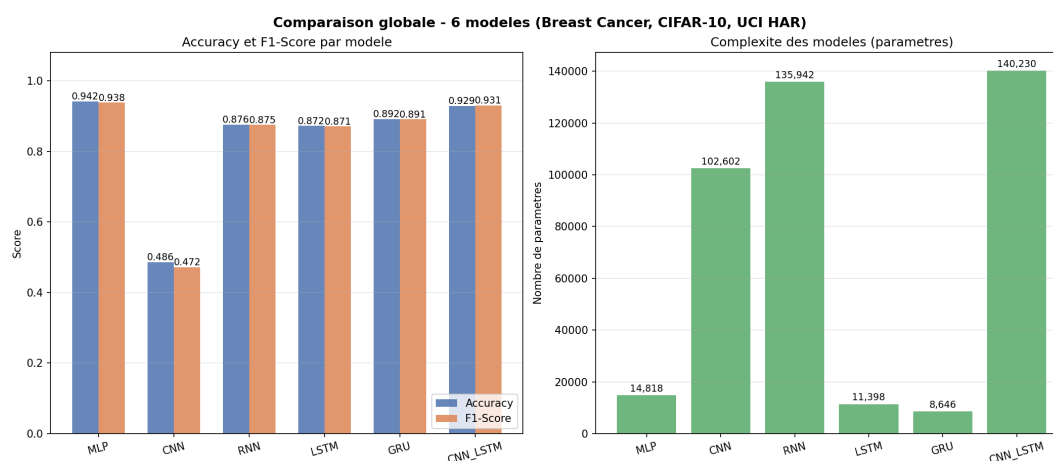


Figure 6 — Comparaison globale des performances et complexités

La comparaison globale met en lumière plusieurs enseignements fondamentaux. Premièrement, l'adéquation entre l'architecture et la nature des données est le facteur déterminant de la performance. Le MLP, architecture la plus simple, atteint 94,19% sur des données tabulaires bien structurées, alors que le CNN, pourtant plus complexe, ne dépasse pas 48,60% sur un sous-ensemble d'images insuffisant. Deuxièmement, sur le même dataset réel UCI HAR, la hiérarchie $RNN (87,61\%) \approx LSTM (87,21\%) < GRU (89,18\%) < CNN+LSTM (92,91\%)$ démontre clairement que les mécanismes de portes et l'hybridation spatiale+temporelle apportent un gain réel. Troisièmement, le nombre de paramètres n'est pas un garant de performance : le RNN avec 135,942 paramètres est surclassé par le GRU avec seulement 8,646 paramètres.

4.2 Adéquation données–modèle

L'analyse des résultats permet de dégager des principes directeurs pour le choix d'architecture en fonction du type de données :

- **Données tabulaires** (Breast Cancer) : le MLP est l'architecture naturelle et la plus efficace. Les données tabulaires ne présentent pas de structure spatiale ou temporelle que les CNN ou RNN pourraient exploiter. Le MLP, par ses connexions denses, capture directement les relations entre les variables.
- **Images** (CIFAR-10) : le CNN est l'architecture appropriée, mais sa performance dépend crucialement de la quantité de données disponibles. Avec un dataset complet (50 000 échantillons), un CNN atteint typiquement plus de 90%. La réduction à un sous-ensemble provoque un surapprentissage sévère, illustrant la *data hunger* des réseaux convolutifs.
- **Séries temporelles** (UCI HAR, données réelles) : les architectures récurrentes sont indispensables. La hiérarchie $RNN \approx LSTM < GRU < CNN+LSTM$ confirme que les mécanismes de portes améliorent la capture des dépendances temporelles. Le GRU, plus léger que le LSTM, converge plus rapidement et atteint ici de meilleures performances (89,18% vs 87,21%).
- **Données spatio-temporelles** (UCI HAR, 9 canaux) : l'architecture hybride CNN+LSTM est le choix optimal. Elle exploite la structure locale (CNN sur les 9 canaux) et la dynamique temporelle (LSTM sur 128 pas de temps) simultanément, démontrant que la combinaison de paradigmes peut surpasser chaque composant isolé (92,91% vs 89,18% pour le GRU seul).

4.3 Analyse des compromis paramètres–performance

Un aspect remarquable de cette étude est le rapport paramètres/performance. Le GRU, avec seulement 8,646 paramètres, atteint 89,18% sur UCI HAR, soit le meilleur ratio paramètres/performance des modèles récurrents. À l'inverse, le CNN mobilise 102,602 paramètres pour seulement 48,60% sur CIFAR-10, soit le pire ratio. Le MLP offre un bon compromis avec 94,19% pour 14,818 paramètres. Le CNN+LSTM, malgré ses 140,230 paramètres, atteint 92,91% sur un problème complexe à 6 classes, justifiant sa complexité par une performance supérieure. Ces observations soulignent que la complexité du modèle doit être calibrée en fonction du problème et des données disponibles, et qu'un modèle plus simple et bien adapté surpasse toujours un modèle complexe mal configuré.

5. Impact sociétal et éthique

5.1 Applications sociétales du deep learning

Les architectures étudiées dans ce travail trouvent des applications dans de nombreux domaines à fort impact sociétal. Le MLP appliqué à la classification de tumeurs illustre l'utilisation du deep learning dans le domaine médical, où des systèmes d'aide au diagnostic peuvent améliorer la détection précoce des cancers et sauver des vies. Les CNN, au cœur des systèmes de vision par ordinateur, alimentent la conduite autonome, la surveillance vidéo et l'imagerie médicale. Les architectures récurrentes (LSTM, GRU) sont essentielles dans la prédiction de séries temporelles financières, la reconnaissance vocale et la traduction automatique. Le dataset UCI HAR utilisé dans ce travail illustre parfaitement une application concrète : la reconnaissance d'activités humaines à partir de smartphones, qui sous-tend les applications de fitness, de santé mobile et de surveillance des personnes âgées. Les architectures hybrides ouvrent la voie à des applications avancées telles que l'analyse vidéo en temps réel, le monitoring de patients ou la détection d'anomalies dans les infrastructures critiques.

5.2 Biais et équité

L'un des défis éthiques majeurs du deep learning réside dans la reproduction et l'amplification des biais présents dans les données d'entraînement. Un modèle entraîné sur des données non représentatives peut produire des prédictions discriminatoires. Par exemple, un MLP de diagnostic médical entraîné principalement sur des données d'une population spécifique pourrait être moins performant, voire dangereux, lorsqu'il est appliqué à d'autres populations. La sous-représentation de certaines catégories démographiques dans les jeux de données médicaux est un problème documenté et persistant. Dans le cas d'UCI HAR, les 30 volontaires âgés de 19 à 48 ans ne représentent qu'une fraction de la population mondiale : un modèle déployé en production devrait être évalué sur des personnes plus âgées, des enfants, et des personnes à mobilité réduite. Il est donc impératif d'auditer les données d'entraînement et d'évaluer les modèles sur des sous-groupes démographiques pour garantir l'équité.

5.3 Transparence et explicabilité

Les réseaux de neurones profonds sont souvent qualifiés de *boîtes noires* en raison de la difficulté à interpréter leurs décisions. Cette opacité pose problème dans les domaines sensibles tels que la santé, la justice ou la finance, où la capacité à expliquer une décision est une exigence réglementaire et éthique. Des techniques telles que LIME, SHAP et les cartes de saillance (saliency maps) permettent d'atténuer partiellement ce problème, mais l'explicabilité complète des modèles profonds reste un sujet de recherche ouvert. Dans notre étude, l'analyse de l'importance des

caractéristiques du MLP offre un début d'interprétation, mais les couches intermédiaires des CNN et des LSTM restent largement opaques.

5.4 Impact environnemental

L'entraînement de modèles de deep learning, en particulier les architectures à grande échelle, consomme des quantités significatives d'énergie électrique et contribue aux émissions de gaz à effet de serre. Un modèle de langage volumineux peut nécessiter l'équivalent de plusieurs dizaines de tonnes de CO₂ pour son entraînement. Même à l'échelle de nos expériences, l'entraînement sur CPU de nos différents modèles a requis plusieurs minutes à heures de calcul. Le choix d'architectures économes en paramètres, comme le GRU par rapport au LSTM, constitue une contribution à la réduction de l'empreinte carbone du deep learning. L'optimisation des hyperparamètres et l'utilisation de techniques d'accélération (quantification, pruning, distillation de connaissances) sont des pistes à explorer pour concilier performance et responsabilité environnementale.

6. Conclusion

6.1 Synthèse des travaux

Ce travail a permis d'explorer systématiquement six architectures de deep learning, du MLP à l'architecture hybride CNN+LSTM, en passant par les CNN, RNN, LSTM et GRU. Chaque modèle a été implémenté en PyTorch, entraîné sur un jeu de données réel adapté à sa nature (Breast Cancer pour le MLP, CIFAR-10 pour le CNN, UCI HAR pour les modèles récurrents et hybrides), et évalué selon des métriques standardisées. Le cadre théorique détaillé a permis de comprendre les fondements mathématiques de chaque architecture, des équations du neurone formel aux mécanismes de portes du LSTM et du GRU.

6.2 Résultats clés

Les résultats expérimentaux confirment plusieurs principes fondamentaux du deep learning :

1. **L'adéquation données-modèle est déterminante** : le MLP excelle sur les données tabulaires (94,19%), les architectures récurrentes sur les séries temporelles réelles (GRU : 89,18%), et l'hybride CNN+LSTM sur les données spatio-temporelles (92,91%).
2. **La quantité de données est critique** : le CNN sur CIFAR-10 réduit (48,60%) illustre dramatiquement l'impact du sous-échantillonnage sur les réseaux convolutifs.
3. **Les mécanismes de portes améliorent la modélisation séquentielle** : sur UCI HAR, la hiérarchie RNN (87,61%) $\approx$ LSTM (87,21%) < GRU (89,18%) démontre que les portes résolvent efficacement le problème du gradient qui s'évanouit.

4. **L'hybridation est puissante** : la combinaison CNN+LSTM atteint 92,91% sur UCI HAR en tirant parti de la complémentarité entre extraction spatiale et modélisation temporelle.
5. **Plus de paramètres ne signifie pas meilleure performance** : le CNN (102,602 paramètres, 48,60%) est surclassé par le GRU (8,646 paramètres, 89,18%).

6.3 Perspectives

Plusieurs pistes de recherche se dégagent de ce travail. L'utilisation de techniques d'augmentation de données pourrait améliorer significativement les performances du CNN sur CIFAR-10. L'implémentation d'architectures plus avancées (ResNet, Attention, Transformer) permettrait de comparer les approches à nos modèles de base. L'intégration de mécanismes d'attention dans les architectures récurrentes (attention additive, attention multiplicative) pourrait encore améliorer les performances sur les séries temporelles longues. Enfin, le déploiement de ces modèles sur des dispositifs embarqués (edge computing) nécessiterait des techniques de compression (quantification, élagage, distillation) pour concilier performance et efficacité computationnelle. L'exploration de ces perspectives contribuera à faire progresser à la fois la compréhension théorique et l'application pratique du deep learning.